QA Walkthrough: Epicenter Pipeline

As of 2026-11-05. Shareable copy: https://claude.ai/artifact/1nLmFSK4HBPZyeazGCYBRb

Purpose

This script accompanies the screen-recorded walkthrough and tells QA how to test the Epicenter **pipeline** feature end to end: developer (directory) layout, GitHub sync, npm build, and the resulting sim. Each step says what to do, what to show on screen, and what counts as a pass. Steps take about 30 minutes total; the build itself runs in under a minute.

Reference project: https://github.com/forio/forio-pipeline-example, the Epicenter React template ("Bike Shop" Excel model) in the developer layout.

Prerequisites

- A GitHub account that can fork `forio/forio-pipeline-example` and add a deploy key to the fork.
- An Epicenter admin login on the test server with author rights on the account you will use (the walkthrough was recorded on `epicenter-sandbox-1.forio.com`, account `wgh`).
- Locally: `git`, `openssl`, `ssh-keygen` (ships with Git for Windows and macOS) and Node 24 if you want to run the optional command-line checks.
- Decide the project short name before you start. The script uses `qa-pipeline-example`.

Vocabulary

Term	Meaning
Developer layout	Epicenter's directory ("rational") layout: the project has `src/`, `target/`, `config/` and `log/`. The git repository is cloned into `src/`; the sim is served from `target/`.
Pipeline	A named, ordered list of operations (`git`, `npm`, `copy`, `clean`, `configure`) that you assemble in the admin UI. Epicenter stores it as `config/pipeline/<name>.cfg1`. Running it produces an audit record and a log.
Deploy key	A read-only SSH key registered on the GitHub repository that lets Epicenter pull it.

Step 1 — Fork the repository and tour the developer layout

Outcome: QA owns a copy of the project on GitHub and understands what Epicenter will clone.

1. On GitHub, open https://github.com/forio/forio-pipeline-example and click **Fork**. Keep the name `forio-pipeline-example`; fork all branches or just `main` (the walkthrough only needs `main`).
2. Clone your fork locally (optional, but useful for showing the files):

```
git clone git@github.com:<your-github-user>/forio-pipeline-example.git
cd forio-pipeline-example
```

3. **Show on screen:** the repository root. Say out loud that the repository root *is* Epicenter's `src/` directory: whatever is at the top level of the repo lands in `src/` on the platform.

Folder	What it is	What the pipeline does with it
`interface/`	The npm project (Vite + React + TypeScript). `package.json` and `package-lock.json` live here.	`npm ci` and `npm run build` run inside this folder; the build writes to `target/interface`.
`model/`	`model.xlsx` (Bike Shop model) and `model.ctx2`.	The `copy` operation copies it to `target/model`.
`pipeline/`	Reference copies of the pipeline definitions (`sync.cfg1`, `build.cfg1`, `build-dev.cfg1`), the same operations you will select in the UI.	Nothing automatic; useful to compare against what the UI saves.
`tests/`, `update_notes/`, `notes/`	Smoke test, screenshots, conversion notes.	Ignored by the pipeline.

4. Open `pipeline/build.cfg1` and read the four operations aloud; you will recreate them in the UI in Step 5.

Pass: the fork exists under your GitHub account and `interface/package-lock.json` is present (the build uses `npm ci`, which fails without a lockfile).

Step 2 — Create the project `qa-pipeline-example`

Outcome: an empty project in the developer layout, ready to receive the repository.

1. Log in to the Epicenter admin UI on the test server and create a new project in your account with these settings:

Setting	Value	API equivalent (`POST /project`)
Short name	`qa-pipeline-example`	`shortName`
Project Access	**Password Protected**	`accessType` (users must log in with a group; not `public`)
Web Access	**Allow access to all URLs by default**	`structure.enforceFilePermissions: false`
Directory Layout	**Developer**	`structure.layout: "RATIONAL"`

2. **Show on screen:** the project's file browser. In the developer layout the project has four top-level folders. Open `src/` and `target/` and show that both are empty apart from the placeholder folders.

Folder	Contents right after creation
`src/`	`data/`, `model/`, `proxy/`, `interface/` (all empty)
`target/`	`data/`, `model/`, `proxy/`, `interface/` with `public/`, `private/`, `facilitator/`
`config/`	`pipeline/` (empty)
`log/`	`model/`, `pipeline/`, `flight/`, `proxy/`, `interface/`

3. Say why Web Access matters: with URL access restricted, an unauthenticated visitor is redirected to a `login.html` that this React app does not have, so the sim would appear broken.

Pass: the project opens, the layout is Developer, and `src/` and `target/` are present and empty.

Command-line alternative: from a clone of the fork with `interface/.env` pointing at your account and project and the credentials file filled in, `cd interface && npm ci && npm run project -- create --private` creates the project with the developer layout, then `npm run project -- web-access allow`.

Step 3 — Connect the project to GitHub

Outcome: Epicenter can pull your fork over SSH with a read-only deploy key.

1. Generate a key pair in a folder that is **not** inside the repository. Epicenter stores the private key as a PKCS#8 key, so generate it with `openssl` (an OpenSSH-format key from `ssh-keygen -t ed25519` is rejected with "Not a base64 encoded stream"):

```
openssl genpkey -algorithm ed25519 -out qa-deploy-key.pem     # private key, PKCS#8 PEM
ssh-keygen -y -f qa-deploy-key.pem > qa-deploy-key.pub        # matching OpenSSH public key
cat qa-deploy-key.pub
```

2. On GitHub open your fork → **Settings** → **Deploy keys** → **Add deploy key**. Title `epicenter-qa`, paste the contents of `qa-deploy-key.pub`, leave **Allow write access** unchecked. **Show on screen:** the key listed as read-only.
3. In the Epicenter admin UI open the project's git integration and enter:

Field	Value
Repository URI	`ssh://git@github.com/<your-github-user>/forio-pipeline-example.git` (the `ssh://` form; the scp-style `git@github.com:user/repo.git` fails with "Illegal character in scheme name")
Algorithm	`ed25519`
Private key	the base64 body of `qa-deploy-key.pem` with the `BEGIN` / `END` lines removed, spec `pkcs8`
Public key	the `ssh-ed25519 AAAA…` line from `qa-deploy-key.pub` (comment optional), spec `openssh`

4. **Show on screen:** the saved integration showing the URI and the public key. The private key is never displayed again.

Pass: the integration saves without error. Nothing is pulled yet; the sync happens in Step 4.

Command-line alternative: `npm run project -- git ssh://git@github.com/<user>/forio-pipeline-example.git qa-deploy-key.pem qa-deploy-key.pub` strips the PEM armor and sends the correct specs for you.

Step 4 — Pipeline 1: sync the repository into `src/`

Outcome: the fork's `main` branch appears under `src/` on the project.

1. In the Epicenter admin UI open the project's **Pipelines** area and create a new pipeline named `sync`. Add one operation:

Operation	Field	Value
Git	Branch	`main`
Git	Confirmed	on (required: the sync discards any local changes in `src/`)
Git	Force	off

   Leave the audit retention at its default (20). Save the pipeline.

2. **Show on screen:** the project file browser now has `config/pipeline/sync.cfg1`. Open it and show it matches `pipeline/sync.cfg1` in the fork: one `git` operation with `"branch": "main"` and `"confirmed": true`.

3. Run the pipeline `sync`. It finishes in a few seconds.

4. **Show on screen:** the audit record going from `RUNNING` to `SUCCEEDED`, then the log `log/pipeline/sync-<auditKey>.log`. The git line reads `Git synchronized on sha(…)`; the SHA matches the latest commit on your fork's `main`.

5. **Show on screen:** `src/` now contains the repository: `interface/`, `model/`, `pipeline/`, `tests/`, `README.md`, a `.git/` folder. `target/` is still empty.

Pass: status `SUCCEEDED`, `src/interface/package.json` exists, `target/interface` has no `index.html` yet.

Step 5 — Pipeline 2: full build with npm

Outcome: the interface is built inside Epicenter's Node container and the model is copied into `target/`.

1. Create a second pipeline named `build` and add four operations in this order:

#	Operation	Field	Value
0	Git	Branch	`main`
0	Git	Confirmed	on
1	NPM	Node version	`v24`
1	NPM	Directory	`interface`
1	NPM	Command	`npm ci` (one command per NPM operation)
1	NPM	Timeout (minutes)	`20`

#	Operation	Field	Value
2	NPM	Node version	`v24`
2	NPM	Directory	`interface`
2	NPM	Command	`npm run build`
2	NPM	Timeout (minutes)	`20`
3	Copy	Location	`MODEL` (leave From and To empty)

Save the pipeline. **Show on screen:** `config/pipeline/build.cfg1` now exists and matches `pipeline/build.cfg1` in the fork.

2. Explain the operations while the audience looks at the pipeline:

#	Operation	What it does	Where
0	`git`	Re-syncs `src/` with `main` (same as Step 4).	platform
1	`npm ci`	Installs dependencies from `package-lock.json` in a Node 24 container. Each `npm` operation runs exactly one command; network access is allowed.	container, working dir `/src/interface`
2	`npm run build`	`tsc -b && vite build`; Vite writes to `/target/interface` (the project's `target/` is mounted there).	container
3	`copy MODEL`	Copies `src/model/` to `target/model/`.	platform

3. Run the pipeline `build`. Expect about 35 seconds: `npm ci` is the long step. **Show on screen:** the audit status `RUNNING`, then `SUCCEEDED`.
4. Open the log `log/pipeline/build-<auditKey>.log`. It is one JSON object per log entry, each with `index` (operation number), `operationType`, `status` and `message`. **Show on screen** and read these lines:
   - `[0 GIT COMPLETED] Git synchronized on sha(…)`
   - `[1 NPM RUNNING] Now using node v24.x` followed by npm's install output, then `[1 NPM COMPLETED] Container completed successfully`
   - `[2 NPM RUNNING] ✓ built in 4.6s` with the list of files written under `../../target/interface/`
   - `[3 COPY COMPLETED] success`
5. Run `build` a second time to show it is repeatable; the audit list now shows two `SUCCEEDED` records.

Pass: status `SUCCEEDED`, all four operations `COMPLETED`, no operation `SKIPPED` (a `SKIPPED` operation means an earlier one failed).

Step 6 — Show the build output and run the sim

Outcome: the files the pipeline produced are in `target/`, and a user can log in and play.

1. **Show on screen:** `target/interface/` now holds `index.html` and an `assets/` folder with the hashed CSS and JS bundles. `target/model/` holds `model.xlsx` and `model.ctx2`. Point out that nothing was uploaded by hand: `target/` is entirely pipeline output.
2. Because the project is Password Protected, the sim needs a group and users. In the admin UI create a group (for example `qa-test`) in `qa-pipeline-example` and add two users: one **facilitator** and one **participant**. Record the credentials somewhere private; never in the repository.
3. Open `https://<server>/app/<account>/qa-pipeline-example/` in a browser. **Show on screen:** the page title "Bike Shop" and the Log In form. Open the browser console and show there are no errors.
4. Log in as the facilitator. The facilitator view creates the first episode automatically. Log out.
5. Log in as the participant. **Show on screen:** the participant view creates a run of `model.xlsx` and displays the Bike Shop variables (price, sales, revenue, costs, profit over time). Step the model and show the values change.

Pass: `target/interface/index.html` and `target/model/model.xlsx` exist; the sim loads at the project URL; both logins work and the participant sees model values.

Command-line alternative: with the group and users listed in the credentials file, `npm run test:smoke` logs in as both roles, creates the episode and run, and prints `Step`, `Price`, `Bike_Sales` and `Profit`.

Checklist, troubleshooting and clean-up

Pass/fail checklist

- ☐ Fork exists and contains `interface/package-lock.json`
- ☐ Project `qa-pipeline-example` created: Password Protected, Web Access allow all URLs, Developer layout
- ☐ `src/` and `target/` shown empty after creation
- ☐ Deploy key listed on the fork as read-only; git integration saved with the `ssh://` URI
- ☐ Pipeline `sync` built in the UI (one Git operation), SUCCEEDED, and `src/` shows the repository files
- ☐ Pipeline `build` built in the UI (Git, NPM `npm ci`, NPM `npm run build`, Copy MODEL), SUCCEEDED with operations 0–3 all COMPLETED; log shown
- ☐ `target/interface/index.html` and `target/model/model.xlsx` present
- ☐ Sim loads at the project URL; facilitator and participant can log in; participant sees model values

Troubleshooting

Symptom (from the pipeline log or UI)	Cause	Fix
`[0 GIT FAILED] Not a base64 encoded stream`	Private key pasted in OpenSSH format or with PEM header lines	Generate with `openssl genpkey`, paste only the base64 body, spec `pkcs8`

Symptom (from the pipeline log or UI)	Cause	Fix
`[0 GIT FAILED] Illegal character in scheme name at index 3`	URI in `git@github.com:user/repo.git` form	Use `ssh://git@github.com/user/repo.git`
Git step fails to authenticate	Deploy key not added to the fork, or added to the upstream repo instead	Add the public key under the fork's Settings → Deploy keys
`[1 NPM FAILED]` during `npm ci`	`package-lock.json` missing or out of date in the fork	Commit a matching lockfile and push; re-run
Build succeeds but `target/interface` stays empty and `src/target/` appears	Vite writing to `../target` relative to `/src/interface`	The template's `vite.config.ts` already writes to `/target/interface` inside the container; make sure the fork has it
Sim URL returns 404 right after creation	Nothing built yet	Run the `build` pipeline
Sim redirects to `login.html`	Web Access is not "allow all URLs"	Change the setting (`structure.enforceFilePermissions: false`)
Operations after a failure show `SKIPPED`	Expected: the pipeline stops at the first failed operation	Fix the failed operation and re-run
A changed pipeline still runs the old operations	The saved `config/pipeline/<name>.cfg1` was not updated	Re-open the pipeline in the UI, check the operations, save again and confirm the file changed

Clean-up after the session

- Delete the project `qa-pipeline-example` (or keep it as a regression fixture).
- Remove the deploy key from the fork and delete `qa-deploy-key.pem`.
- Delete the test users if they were created only for the walkthrough.

Sources

- Repository and CLIs: https://github.com/forio/forio-pipeline-example (see `README.md`, `pipeline/README.md`, `update_notes/2026-11-05-pipeline-conversion.md`).
- Behaviours in the troubleshooting table were observed on `epicenter-sandbox-1.forio.com` (Epicenter 6.1.0) on 2026-11-05.